Checkpoint 3 Report

Silas Wright, Ashmit Mittal, Somto Umeh

Introduction

This report outlines the design and implementation of a full-scale compiler for the C-Minus (C-) language. The compiler is a multi-pass system that transforms source code into TM (Tiny Machine) assembly language. Developed over three distinct checkpoints, the project integrates lexical analysis, syntax parsing, semantic validation, and target code generation using JFlex, CUP, and Java.

The objective was to create a robust tool capable of not only translating valid programs but also providing meaningful error feedback and maintaining architectural integrity across the front-end and back-end phases.

Overall Design Architecture

The compiler follows a traditional pipeline architecture, divided into several logical phases:

Lexical and Syntactic Analysis

Using JFlex, we defined the lexical rules for C-, including keywords (e.g., if, while, int), operators, and identifiers. The parser, generated by CUP, consumes these tokens to build an Abstract Syntax Tree (AST). The grammar was designed to handle the recursive nature of C- expressions and statements while resolving ambiguities like the "dangling else" through precedence declarations.

Semantic Analysis and Symbol Table

The Semantic Analyzer performs a post-order traversal of the AST. Its primary role is to enforce scope rules and type consistency. It utilizes a Symbol Table implemented as a stack of hash maps to represent nested scopes (global, function, and block).

Intermediate Representation (AST)

The AST serves as the primary intermediate representation. It is composed of nodes representing various language constructs, such as VarDeclExp for declarations and OpExp for arithmetic or logical operations.

Code Generation

The final phase, CodeGenerator, traverses the AST to emit TM assembly instructions. This phase manages register allocation, memory layout for global and local variables, and the creation of activation records for function calls.

Implementation Process and Significant Changes

The transition from Checkpoint 2 (Semantic Analysis) to Checkpoint 3 (Code Generation) necessitated several significant modifications including refactoring the visitor interface and implementing code generation for control structures.

Technical Implementation Details

Translation to Assembly

The CodeGenerator stored a frame pointer (FP) and managed local variables and parameters using offsets. When a function is called, the caller pushes arguments onto the stack. The callee then saves the return address and sets up its own frame. Since TM uses relative jumps, and the target address for a forward jump (like an if or while loop) is unknown when the instruction is first emitted, we implemented a patchAbsJump mechanism in the TMEmitter class.

Error Handling and Repair

We prioritized informative error messages. The parser reports errors with line and column numbers. In semantic analysis, we avoid "cascading errors" by assigning a default type (usually int) to invalid expressions, allowing the analyzer to proceed without crashing.

Array Handling

Arrays in C- are passed by reference. The compiler distinguishes between a scalar variable and an array base address. For indexed access (x[i]), the code generator calculates the effective address by adding the index to the base pointer, including a mandatory lower-bound check to halt the TM simulator if the index is negative.

Assumptions and Limitations

Assumptions:

1. Integer Size: We assume all integers and boolean values occupy exactly one memory word in the TM simulator.
2. Memory Limit: The TM simulator has a finite memory space; we assume the stack and global data will not collide during execution for the provided test cases.

Limitations:

1. No Optimization: The current implementation does not perform constant folding or dead code elimination.
2. Limited Type System: Only int, bool, and void are supported. There is no support for floating-point numbers or strings.

Possible Improvements

To enhance the compiler, the most important missing phase of this compiler is the lack of instruction optimization. By implementing a peephole optimizer we could remove redundant LD (load) and ST (store) instructions. Another major improvement would be support for multidimensional arrays and arrays of variable length. Additionally, future work could include expanding the C- language itself and building out support for the expanded version would prove to greatly increase the power of our compiler.

Contributions and Lessons Learned

Team Contributions

This project was completed almost exclusively during group meetings and peer programming sessions. Our team's collaborative experience and problem solving skills were required to navigate the challenges we came across during development. However, we have listed each member's main contributions as well.

- Somto Umeh: Led the creation of the CodeGenerator class authoring the majority of its functions.
- Silas Wright: Developed the TMEmitter class and contributed to the emitter functions in the CodeGenerator class, primary writer for the report.
- Ashmit Mittal: Focused on initial refactoring, understanding system requirements and building out structure. Wrote the helper classes FunctionInfo and VarInfo and documented README.

Lessons Learned

The implementation of the CodeGenerator was the most challenging phase. It taught us the critical importance of a well-defined Calling Convention. Small offsets in stack frame management could lead to catastrophic "memory leaks" or corrupted return addresses, which were difficult to debug in the low-level TM environment. We also learned that a clean AST is vital; the more work done during semantic analysis (such as type decoration), the simpler the code generation logic becomes.

Conclusion

The C- compiler successfully implements the full lifecycle of compilation, from raw source text to executable TM assembly. By adhering to a modular design and making the most of powerful tools like JFlex and CUP, we created a system that is both extensible and maintainable. The resulting compiler is a functional tool capable of executing complex logic, including recursion and array manipulation, within the Tiny Machine environment.